



TREMPIST - טרמפיסט

מסמך אפיון

שני גל

גרסה 1.0

12.11.2025

היסטוריית גרסאות המסמך

תאריך	גרסה	תקציר השינויים
12.11.2025	1	מסמך ראשוני

מסמך אפיון פרויקט



1. הקדמה

1.1 מטרה

מטרת המסמך היא להגדיר באופן ברור ומפורט את הדרישות, המאפיינים והתפקודים של מערכת **TREMPIST**, כדי לשמש בסיס לפיתוח, בדיקות, עיצוב והטמעה של האפליקציה. המסמך נועד לשמש כלי עבודה למפתחי המערכת, למנהלי הפרויקט, למעצבי הממשק ולבודקי האיכות, וכן מקור מידע לכל גורם המעורב בתהליך הפיתוח.

1.2 המוצר

שם המוצר: **TREMPIST – טרמפיסט**
האפליקציה מאפשרת שיתוף טרמפים בין אנשים – אך ורק בין מי שמוכנים גם לתת וגם לקבל טרמפ. מדובר במערכת קהילתית המבוססת על הדדיות, שוויון וביטחון משתמשים. האפליקציה כוללת:

- הרשמה ויצירת פרופיל משתמש (נהג/נוסע).
- חיפוש והתאמה בזמן אמת בין נהגים לנוסעים.
- דירוגים וביקורות להגברת אמון.
- תצוגת מפה מבוססת GPS.
- ממשק משתמש אינטואיטיבי ונגיש.

יתרונות המוצר: קהילתיות אמיתית, ללא תשלום, הפחתת עומסי תנועה וזיהום סביבתי, קידום אחריות הדדית.

1.3 הגדרות

- **טרמפיסט:** משתמש באפליקציה, יכול לשמש כנהג או כנוסע.
- **טרמפ:** נסיעה משותפת ללא תשלום בין שני משתמשים באפליקציה.
- **קרדיט:** יחידת ערך באפליקציה. כל טרמפ מקנה לנהג **קרדיט 1**, ונדרש **קרדיט 1** כדי להזמין טרמפ כנוסע. כך נשמרת הדדיות – רק מי שנותן טרמפים יכול גם לקבל.
- **GPS:** מערכת מיקום עולמית לתיאום נסיעות בזמן אמת.

מסמך אפיון פרויקט

- **דירוג:** חוות דעת של משתמש על משתמש אחר לאחר נסיעה, לצורך חיזוק אמון ובטיחות קהילתית.

1.4 תקציר

המסמך כולל שלושה פרקים עיקריים:

1. **הקדמה** – מציגה את מטרות המסמך והפרויקט.
2. **תיאור כללי** – מתאר את מאפייני המערכת, המשתמשים והאילוצים.
3. **דרישות מפורטות** – מפרטת את כל הדרישות הטכניות, הפונקציונליות והעיצוביות של המערכת.

2. תיאור כללי

2.1 פונקציונליות

- ★ הרשמה, התחברות וניהול פרופיל אישי.
- ★ אפשרות לפרסם טרמפ או להצטרף לטרמפ קיים.
- ★ מנגנון קרדיטים – כל משתמש שמעניק טרמפ מקבל קרדיט , וכל טרמפ כנוסע דורש קרדיט. כך נשמרת מערכת הדדית המעודדת נתינה לצד קבלה.
- ★ מערכת התאמה חכמה בזמן אמת בין נהגים ונוסעים.
- ★ תצוגת מפה מבוססת Google Maps API להצגת מסלול, מיקום נהג ונוסע.
- ★ מערכת דירוגים וביקורות בין משתמשים.
- ★ ממשק ניהול (Admin) לניטור, בקרה ודיווח.
- ★

2.2 קהל היעד

האפליקציה מיועדת לנהגים ונוסעים בגילאי **17 ומעלה**, כולל **נהגים חדשים ותלמידי כיתות י"א–י"ב**, המעוניינים בפתרון תחבורתי חברתי, יעיל וחינמי.

קהל היעד כולל תלמידים, סטודנטים, עובדים ותושבי פריפריה, המשתמשים בנסיעות יומיומיות לבית הספר, לאוניברסיטה או לעבודה.

מנגנון הקרדיטים נועד להבטיח שימוש הוגן במערכת – כך שהקהילה מבוססת על **הדדיות ותרומה הדדית**: כל מי שמעוניין לנסוע נדרש גם לשתף טרמפ עם אחרים.

2.3 אילוצים עיקריים

- נדרש חיבור אינטרנט פעיל בכל עת.
- האפליקציה תתמוך במערכות Android ו-iOS.
- ניהול מאגר קרדיטים מאובטח בבסיס הנתונים.
- שמירה על איזון קרדיטים בין משתמשים למניעת ניצול (לא ניתן לרדת מתחת ל-0 קרדיטים).

מסמך אפיון פרויקט

- שימוש ב-Google Maps API.
- המערכת צריכה לתמוך במספר משתמשים רב
- זמני תגובה מהירים במערכת ההתאמה ובחישוב הקרדיטים.

2.4 הנחות ותלויות

האפליקציה תפותח תחילה עבור מערכת ההפעלה Android, באמצעות סביבת הפיתוח Android Studio. לפיכך, בשלב הראשון רק משתמשי Android יוכלו להוריד ולהשתמש באפליקציה.

בהמשך יישקל מעבר לשפת הפיתוח Flutter של Google, אשר תאפשר תמיכה גם במכשירי iOS, לצורך הרחבת קהל המשתמשים.

בנוסף, נניח כי:

- השרת המרכזי ירוץ על סביבת ענן (כגון Google Cloud או AWS) ויתקשר עם האפליקציה דרך REST API מאובטח.
- מנגנון ניהול הקרדיטים יפעל בצד השרת, ויעדכן באופן אוטומטי את מאזן הקרדיטים של כל משתמש עם סיום טרמפ מאושר.
- האפליקציה תעשה שימוש ב-Google Maps API לצורך הצגת מיקום, חיפוש מסלולים ותיאום טרמפים בזמן אמת.
- המערכת תתמוך בשפה העברית כברירת מחדל, עם אפשרות להוספת שפות נוספות בעתיד.
- כל שינוי מהותי בטכנולוגיית הפיתוח, בשירותי המפה או במודל הקרדיטים ידרוש עדכון מחודש של מסמך האפיון והעיצוב.

2.5 סיקור מצב השוק כיום (אופציונלי)

קיימות אפליקציות לשיתוף נסיעות (כגון BlaBlaCar, Waze Carpool), אך רובן לא צלחו, והן מבוססות על תשלום כספי, מה שמחליש את הערך הקהילתי. **TREMPIST** מתבלטת בכך שהיא מבוססת על **כלכלה קהילתית** של קרדיטים, ללא כסף – כל טרמפ שווה קרדיט אחד, מה שמבטיח שיתוף אמיתי, הוגן ושוויוני. המודל שואב השראה מקהילות מבוססות שיתוף (כמו Reddit או Freecycle) שבהן תרומה פעילה מחזקת את השייכות והאמון ההדדי

3. דרישות מפורטות

3.1 דרישות פונקציונליות

דרישה 1: הרשמה והתחברות

- קלט: דוא"ל, סיסמה, מספר טלפון.
- תהליך: אימות זהות ואישור משתמש.
- פלט: גישה לחשבון האישי.

דרישה 2: פרסום טרמפ

- קלט: יעד, שעת יציאה, מספר מקומות פנויים.
- פלט: הטרמפ מוצג לחיפוש פומבי.

דרישה 3: חיפוש טרמפ

- קלט: יעד ומועד נסיעה.
- פלט: רשימת טרמפים זמינים ותוצאות ממוינות לפי קרבה וזמן.

דרישה 4: מערכת דירוגים

- תהליך: לאחר נסיעה, משתמש מדרג את הנהג/נוסע.
- פלט: ציון ממוצע בפרופיל המשתמש.

דרישה 5: ניהול אדמין

- תהליך: ניטור דיווחים, חסימת משתמשים לא הולמים, עדכוני מערכת.

3.2 דרישות של ממשקים חיצוניים

3.2.1 ממשקי משתמש

- ממשק המשתמש יהיה פשוט, אינטואיטיבי ונגיש לכל סוגי המשתמשים (נהגים ונוסעים).
- במסך הראשי יוצגו אפשרויות עיקריות: "מצא טרמפ", "הצע טרמפ", "הודעות", ו"פרופיל אישי".
- הממשק יכלול מפת מיקום בזמן אמת (Google Maps API).
- תתאפשר גישה מהירה לפרטי המשתמש, היסטוריית נסיעות, והודעות.
- יוצגו הוראות שימוש קצרות והנחיות בטיחות.
- תתאפשר עריכת פרטים אישיים והעלאת תמונת פרופיל.
- הדגש הוא על חוויית משתמש נעימה וברורה, תוך שמירה על נגישות מלאה גם למשתמשים חדשים.

3.2.2 ממשקי חומרה

האפליקציה תשתמש בחומרת המכשיר לצורך:

- שימוש ב-GPS של הטלפון לזיהוי מיקום בזמן אמת.
- גישה לרשת הסלולרית/Wi-Fi לצורך תקשורת עם השרת.
- שימוש במצלמה לצורך העלאת תמונת פרופיל (אופציונלי).
- שימוש ברטט/התראה קולית לצורך קבלת הודעות או אישורי נסיעה.

3.2.3 ממשקי תוכנה

צד הלקוח: אפליקציית המובייל (Android/iOS) שבה המשתמשים מבצעים הרשמה, פרסום או חיפוש טרמפים, ומנהלים את הפרופיל שלהם.

צד השרת : מערכת מרכזית המעבדת את הבקשות, שומרת נתוני משתמשים ונסיעות, מנהלת דירוגים ומבצעת התאמות בין נהגים לנוסעים.
התקשורת בין שני הצדדים תתבצע בפרוטוקול HTTPS ובאמצעות REST API.

3.2.4 ממשקי תקשורת (למשל תקשורת בין שרת ללקוחות)

המערכת תאפשר תקשורת רציפה בין השרת ללקוחות בזמן אמת:

- הלקוח ישלח לשרת בקשות כגון "חפש טרמפ", "הצע טרמפ", "עדכן מיקום".
- השרת יחזיר תגובות הכוללות תוצאות חיפוש, פרטי התאמות, הודעות, ודירוגים.
- תקשורת המיקום תתבצע באמצעות Google Maps API לשם הצגת נקודות איסוף ויעד מדויקות.
- העברת הנתונים תעשה בפרוטוקול מאובטח (HTTPS), עם הצפנת נתונים רגישים (כגון מיקום וזיהוי משתמש).

3.3 דרישות לא פונקציונליות

- ביצועים: זמן תגובה ≥ 2 שניות.
- מהימנות: זמינות 99%.
- אבטחה: הצפנת נתונים (SSL).
- ניידות: תאימות ל-Android ול-iOS.
- תחזוקה: קוד מתועד ומודולרי.

3.4 דרישות בסיס נתונים

- נתונים שיישמרו: פרטי משתמשים, טרמפים, דירוגים, היסטוריית נסיעות.
- טכנולוגיה: SQL / Firebase.
- אבטחה: גישה לפי הרשאות, הצפנת סיסמאות.

3.5 דרישות נוספות

- שילוב מערכת הודעות פנימית בין משתמשים.
- אפשרות לשיתוף נסיעה ב-WhatsApp או SMS.
- תמיכה במצב כהה (Dark Mode).

4. תכנון

4.1 תכנון וניהול לוח זמנים לפיתוח המערכת

שלב	תיאור	תאריך יעד
אפיון ותכנון	כתיבת מסמך אפיון ואישורו	15.11.25
עיצוב UI ו-UX	עיצוב מסכים ואפיון זרימות משתמש	31.11.25
פיתוח צד לקוח	בניית אפליקציה ב-flutter	31.1.26
פיתוח צד שרת	פיתוח API ובסיס נתונים	31.2.26
בדיקות מערכת	QA ו Testing	1.4.26
השקה	גרסה ראשונה ל-App Store/Google Play	15.4.26

4.2 ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם

סיכון	השפעה	דרך התמודדות
בעיות אבטחה	בינונית	שימוש בפרוטוקולי HTTPS ואימות דו-שלבי
עומס משתמשים	גבוהה	הקמת שרתים גמישים בענן
עיכובים בלוח	בינונית	חלוקה מדויקת למשימות
קושי באימוץ משתמשים	נמוכה	קמפיין קהילתי ושיווק